JAVA COLLECTIONS FRAMEWORK

Core interfaces • Implementations • Selection guide • Coding patterns

1. What is the Java Collections Framework (JCF)?

The **Java Collections Framework** is a set of interfaces, classes, and algorithms used to store and process groups of objects. It provides reusable data structures instead of requiring us to build every structure from scratch. Most core collection types are in `java.util`.

The framework has three important parts

• **Interfaces:** contracts describing behavior, such as List, Set, Queue, Deque, and Map.

• **Implementations:** concrete classes such as ArrayList, HashSet, ArrayDeque, and HashMap.

• **Algorithms/utilities:** methods in Collections, such as sort, reverse, shuffle, and binarySearch.

High-level structure

```
Iterable
  ■■ Collection
      ■■ List       → ArrayList, LinkedList
      ■■ Set        → HashSet, LinkedHashSet, TreeSet
      ■■ Queue      → PriorityQueue, Deque
                        ■■ ArrayDeque, LinkedList

Map (separate from Collection)
  ■■ HashMap
  ■■ LinkedHashMap
  ■■ TreeMap
```

Key distinction: Map is part of the Collections Framework, but Map does **not** extend the Collection interface. A List stores elements; a Map stores key–value mappings.

2. Choosing the right structure

Need	Good starting choice	Why
Indexed access, duplicates allowed	ArrayList	Fast get(index), good general-purpose list
Unique values	HashSet	Fast average membership checks
Key → value lookup	HashMap	Fast average lookup by key
FIFO processing	ArrayDeque	Add at tail, remove at head
Priority-based retrieval	PriorityQueue	Head is the least element by default
Sorted unique elements	TreeSet	Maintains sorted order

3. List, Set, and Map

List — ordered sequence

A List preserves positional order and allows duplicates. ArrayList is the usual default; LinkedList is a doubly linked list and also implements Deque.

```
List<String> names = new ArrayList<>();
names.add("Anu");
names.add("Bala");
names.add("Anu");          // duplicates allowed
System.out.println(names.get(1)); // Bala
names.set(0, "Asha");
names.remove("Bala");
```

Set — unique elements

A Set does not allow duplicate elements. HashSet gives no guaranteed iteration order; LinkedHashSet preserves insertion order; TreeSet keeps elements sorted.

```
Set<Integer> values = new HashSet<>();
values.add(10);
values.add(10); // duplicate ignored
values.add(4);
System.out.println(values.contains(4)); // true
```

Map — keys mapped to values

Keys are unique; a repeated key replaces its previous value. HashMap does not guarantee order. Use getOrDefault for counters.

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Kavi", 80);
scores.put("Kavi", 95); // replaces 80
int score = scores.getOrDefault("Ravi", 0);
for (Map.Entry<String, Integer> e : scores.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

4. Queue, Deque, and PriorityQueue

Queue commonly represents FIFO processing: first in, first out. Deque supports operations at both ends and can be used as a queue or stack. PriorityQueue retrieves elements by priority rather than insertion time.

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(10); q.offer(20);
System.out.println(q.peek()); // 10
System.out.println(q.poll()); // removes and returns 10

Deque<Integer> dq = new ArrayDeque<>();
dq.push(5); dq.push(8);
System.out.println(dq.pop()); // 8

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30); pq.offer(10); pq.offer(20);
System.out.println(pq.poll()); // 10 (minimum first)
```

Method pairs to remember

• **Queue:** offer adds, poll removes head (null if empty), peek reads head (null if empty).

• **Deque/stack style:** push adds at front, pop removes front, peek reads front.

• **PriorityQueue:** default is min-heap behavior; use a Comparator for a different priority.

5. Common methods and iteration

Operation	Typical method	Notes
Add an element	add(e) / offer(e)	offer is commonly used with Queue
Remove element	remove(e) / poll()	poll removes queue head; remove(e) removes a matching element
Check membership	contains(e)	Usually fast in HashSet; linear in ArrayList
Count elements	size()	Returns number of elements/mappings
Empty check	isEmpty()	Prefer over size() == 0 for readability
Clear collection	clear()	Removes all elements
Sort a List	Collections.sort(list)	Natural order; elements must be comparable

Iteration examples

```
List<Integer> nums = new ArrayList<>(List.of(3, 1, 2));
for (int x : nums) System.out.println(x);

Collections.sort(nums);       // [1, 2, 3]
Collections.reverse(nums);    // [3, 2, 1]

Map<String, Integer> m = new HashMap<>();
m.put("A", 1); m.put("B", 2);
for (Map.Entry<String, Integer> e : m.entrySet()) {
    System.out.println(e.getKey() + "=" + e.getValue());
}
```

Complexity snapshot (typical)

Structure	Common operation	Typical cost
ArrayList	get(index) / append	O(1) / amortized O(1)
ArrayList	Insert/remove middle	O(n)
HashSet / HashMap	add / contains / get	Expected O(1)
TreeSet / TreeMap	add / lookup	O(log n)
PriorityQueue	offer / poll	O(log n)

Costs depend on implementation and operation; these are common interview-level expectations, not universal guarantees.

6. Problem 1 — Remove duplicates while preserving order

Problem: Given a sequence of integers, print each value only once, keeping the first occurrence order.

Sample input

```
7
4 2 4 3 2 5 3
```

Sample output

```
4 2 3 5
```

Java solution

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Set<Integer> seen = new LinkedHashSet<>();
        for (int i = 0; i < n; i++) seen.add(sc.nextInt());
        for (int x : seen) System.out.print(x + " ");
        sc.close();
    }
}
```

Why LinkedHashSet? It rejects duplicates and preserves insertion order. A HashSet removes duplicates too, but output order is not guaranteed.

Complexity: Expected O(n) time and O(k) space, where k is the number of unique values.

7. Problem 2 — Balanced parentheses using a Deque

Problem: Given a string containing only '(' and ')', determine whether parentheses are balanced. Each closing bracket must match an earlier opening bracket.

Sample input / output

```
(()())  → true
())(    → false
```

Java solution

```
import java.util.*;

public class Main {
    static boolean balanced(String text) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char ch : text.toCharArray()) {
            if (ch == '(') stack.push(ch);
            else if (ch == ')') {
                if (stack.isEmpty()) return false;
                stack.pop();
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(balanced("(()())")); // true
        System.out.println(balanced("())("));   // false
    }
}
```

Complexity: O(n) time and O(n) worst-case extra space. ArrayDeque is generally preferred over the legacy Stack class for stack operations.

8. Common mistakes & interview revision

• **Prefer interface declarations:** List names = new ArrayList<>(); keeps code flexible.

• **Order matters:** use LinkedHashSet/LinkedHashMap for predictable iteration and TreeSet/TreeMap for sorted elements/keys.

• **poll() vs remove():** poll() returns null when a queue is empty; remove() throws an exception.

• **PriorityQueue:** only the head is guaranteed to be the least element by default; iteration is not sorted.

• **Modification during iteration:** structural changes may trigger ConcurrentModificationException. Use Iterator.remove() when appropriate.

• **Primitive types:** use Integer, Double, Character rather than int, double, char as generic type arguments.

Quick selection guide

Requirement	Choose
Duplicates + index access	ArrayList
Unique values, no order needed	HashSet
Unique values in insertion order	LinkedHashSet
Unique values in sorted order	TreeSet
Key-value lookup	HashMap
FIFO queue or stack	ArrayDeque
Retrieve by priority	PriorityQueue with suitable Comparator

Interview Aspect — concise answer

"The Java Collections Framework provides interfaces and implementations for reusable data structures. List allows duplicates and positional access; Set enforces uniqueness; Queue models processing order; and Map stores key–value mappings but is not a subtype of Collection. I choose an implementation based on ordering, lookup requirements, and operation complexity."

Practice questions

• Find the first non-repeating character using a Map.

• Use a PriorityQueue to print the k smallest numbers.

• Reverse a list using Collections.reverse().

• Compare ArrayList and LinkedList for indexed access and insertion.

• Implement a word-frequency counter using HashMap.

Revision checklist

■ Explain List / Set / Queue / Map ■ Choose HashSet vs LinkedHashSet vs TreeSet ■ Use ArrayDeque as stack and queue ■ Know common methods ■ Explain typical time complexities.